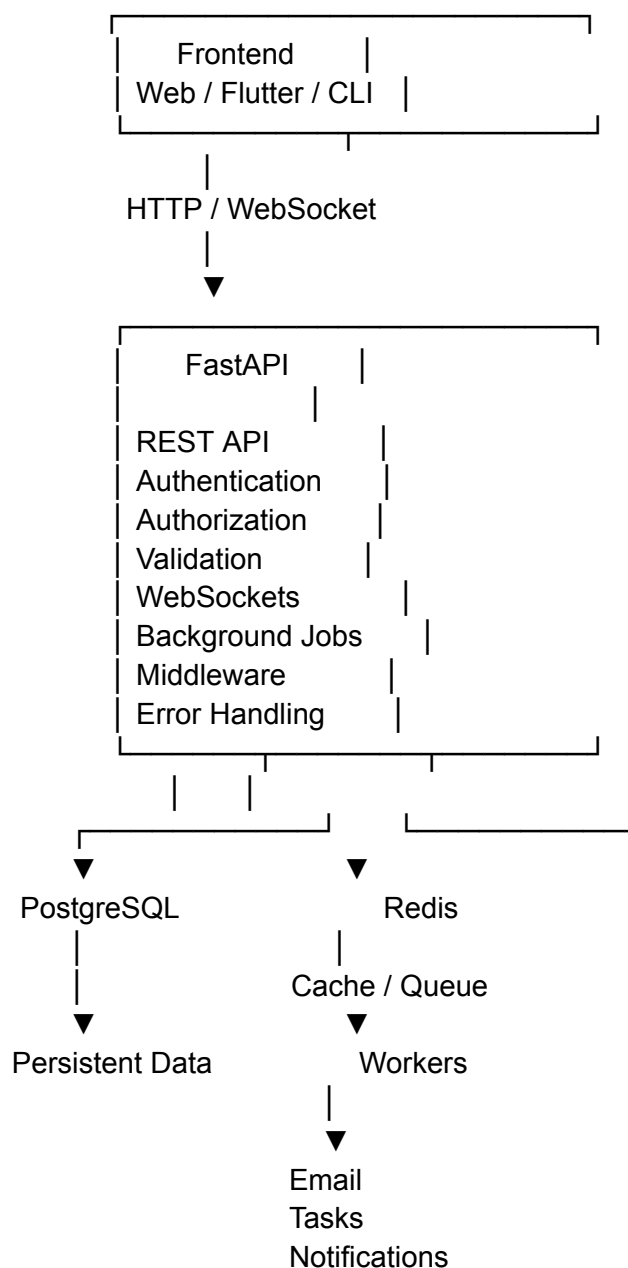


PulseBoard — Complete FastAPI Mastery Roadmap

What you are building

PulseBoard is a multi-tenant incident and task-management backend.

By the end, you will have:



PHASE 0 — Understand the Architecture Before Coding

Do not write application code yet.

First understand what you're building.

Your system has five major layers:

API Layer
↓
Service Layer
↓
Repository/Data Access Layer
↓
Database

with supporting infrastructure:

Authentication
Authorization
Caching
Background jobs
WebSockets
File storage
Logging

Request flow

A request such as:

POST /api/v1/projects/10/tasks

should conceptually go through:

HTTP Request
↓
Middleware
↓
Authentication
↓
Organization check
↓
Permission check
↓
Pydantic validation
↓

API endpoint
↓
Service
↓
Repository
↓
PostgreSQL
↓
Response model
↓
HTTP Response

You should understand this flow before proceeding.

PHASE 1 — Set Up the Project

Step 1. Create the repository

Create:

pulseboard/

Initialize Git.

Create:

README.md

.gitignore

.env.example

pyproject.toml

Do **not** commit `.env`.

Step 2. Create the Python environment

Use a modern Python version.

Install:

fastapi

uvicorn

pydantic
pydantic-settings
sqlalchemy
asyncpg
alembic
python-jose
passlib
bcrypt
python-multipart
httpx
pytest
pytest-asyncio

Later you'll add:

redis
celery

or another worker framework.

Step 3. Create the initial structure

Start with:

```
pulseboard/  
├── app/  
│   ├── main.py  
│   ├── api/  
│   ├── core/  
│   ├── db/  
│   ├── models/  
│   ├── schemas/  
│   ├── services/  
│   └── repositories/  
├── tests/  
├── .env.example  
├── .gitignore  
├── pyproject.toml  
└── README.md
```

At this point:

Goal:

uvicorn app.main:app --reload

should start successfully.

PHASE 2 — Your First FastAPI Application

Step 4. Create `main.py`

Create your FastAPI application.

Learn:

- `FastAPI()`
- application metadata
- title
- description
- version
- routes

You should be able to access:

/docs
/redoc
/openapi.json

Step 5. Create your first endpoint

Create:

GET /

Return:

```
{  
  "message": "PulseBoard API"  
}
```

Then create:

GET /health

Return:

```
{  
  "status": "healthy"  
}
```

Step 6. Learn path parameters

Create:

GET /users/{user_id}

Learn:

user_id: int

Experiment with invalid input.

For example:

/users/abc

Observe FastAPI's automatic validation.

Step 7. Learn query parameters

Create:

GET /tasks

with:

/tasks?page=1

/tasks?limit=20

/tasks?status=completed

Learn:

- Required query parameters
- Optional parameters
- Default values
- Type validation

PHASE 3 — Pydantic Deep Dive

This phase is extremely important.

Step 8. Create request schemas

Create:

UserCreate
TaskCreate
TaskUpdate
ProjectCreate

Example:

TaskCreate
├── title
├── description
├── priority
└── due_date

Learn:

- `BaseModel`
- optional fields
- default values
- nested models
- lists
- dictionaries
- enums

Step 9. Add validation

Rules:

title → minimum 3 characters
title → maximum 200 characters
priority → valid enum
due_date → valid datetime

Experiment with:

Field()
field_validator()
model_validator()

Understand **why validation belongs at the API boundary**.

Step 10. Create response models

Create:

TaskResponse
UserResponse
ProjectResponse

Do not return database models directly.

Understand:

Request model
≠
Database model
≠
Response model

This distinction becomes extremely important later.

PHASE 4 — Database

Now introduce PostgreSQL.

Step 11. Start PostgreSQL

Use Docker.

Create:

docker-compose.yml

Initially:

PostgreSQL

only.

Connect FastAPI to it.

Step 12. Learn SQLAlchemy

Create your database configuration.

Learn:

Engine
Session
AsyncEngine
AsyncSession
DeclarativeBase

Use SQLAlchemy 2.x style.

Step 13. Create the User model

Database:

users

Fields:

id
email
password_hash
first_name
last_name
is_active
created_at
updated_at

Learn:

- Primary keys
- Unique constraints
- indexes
- timestamps
- relationships

Step 14. Create database migrations

Configure Alembic.

Run:

alembic revision

alembic upgrade head

Learn:

Migration

Revision

Upgrade

Downgrade

Autogenerate

From now on, **never manually change production schemas**.

Step 15. Build database dependency

Create:

get_db()

Your endpoints should receive:

AsyncSession

through FastAPI dependency injection.

This is your first serious use of:

FastAPI Dependency Injection

PHASE 5 — CRUD

Now build actual application functionality.

Step 16. User CRUD

Create:

POST /users
GET /users
GET /users/{id}
PATCH /users/{id}
DELETE /users/{id}

Learn:

- POST
 - GET
 - PATCH
 - DELETE
 - status codes
 - response models
 - database transactions
-

Step 17. Create projects

Database:

projects

Relationships:

User



Endpoints:

POST /projects
GET /projects
GET /projects/{id}
PATCH /projects/{id}
DELETE /projects/{id}

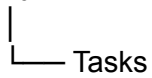
Step 18. Create tasks

Database:

tasks

Relationship:

Project



Task fields:

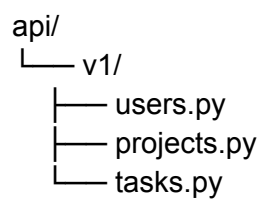
id
title
description
status
priority
project_id
creator_id
assignee_id
due_date
created_at
updated_at

PHASE 6 — Routers

Your application is now getting large.

Don't put everything in `main.py`.

Create:



Learn:

`APIRouter()`

and:

`app.include_router(...)`

Use tags:

Users

PHASE 7 — Service Layer

Now deliberately separate business logic from HTTP.

Instead of:

Endpoint
↓
Database

create:

Endpoint
↓
Service
↓
Repository
↓
Database

Step 19. Create TaskService

Example responsibilities:

```
create_task()
update_task()
assign_task()
complete_task()
delete_task()
```

The API endpoint should primarily handle:

HTTP
↓
Validation
↓
Service
↓
Response

Step 20. Create repositories

Create:

TaskRepository
ProjectRepository
UserRepository

Repositories handle database operations.

For example:

TaskRepository.get_by_id()
TaskRepository.create()
TaskRepository.update()
TaskRepository.delete()

Now you're learning architecture rather than merely syntax.

PHASE 8 — Authentication

This is a major milestone.

Step 21. Password hashing

Never store:

password

Store:

password_hash

Learn:

- hashing
 - salts
 - verification
 - password security
-

Step 22. Registration

Create:

POST /api/v1/auth/register

Process:

Request

↓

Validate email

↓

Validate password

↓

Check existing user

↓

Hash password

↓

Create user

↓

Return response

Step 23. Login

Create:

POST /api/v1/auth/login

Learn OAuth2 password flow.

Return:

```
{  
  "access_token": "...",  
  "token_type": "bearer"  
}
```

Step 24. JWT

Learn:

JWT header

JWT payload

JWT signature

expiration

subject

claims

Implement:

```
create_access_token()
decode_access_token()
```

Step 25. Current user dependency

Create:

```
get_current_user()
```

Then:

GET /users/me

should automatically identify the logged-in user.

This is one of the most important FastAPI dependency patterns.

PHASE 9 — Refresh Tokens

Access tokens should be short-lived.

Implement:

Access Token

↓

short lifetime

Refresh Token

↓

longer lifetime

Endpoints:

POST /auth/refresh

POST /auth/logout

Store refresh-token information in the database.

PHASE 10 — Organizations

Now convert your application into a **multi-tenant system**.

Create:

organizations
organization_members

Relationship:

Organization

- Users
- Projects
- Tasks
- Incidents

Step 26. Organization creation

POST /organizations

The creator becomes:

ORG_ADMIN

Step 27. Organization membership

Create:

POST /organizations/{id}/members
DELETE /organizations/{id}/members/{user_id}
GET /organizations/{id}/members

PHASE 11 — Authorization

Create roles:

SUPER_ADMIN
ORG_ADMIN
MANAGER

MEMBER
VIEWER

Create dependencies:

require_authenticated_user()
require_role()
require_permission()

Example:

DELETE /projects/{id}

might require:

ORG_ADMIN

Step 28. Cross-tenant security

This is a critical challenge.

Suppose:

Organization A
Project 123

and:

Organization B
User Bob

Bob must **never** be able to access:

GET /projects/123

even if he knows the ID.

Every resource access must verify organization membership.

This is one of the most important real-world backend security problems you'll implement.

PHASE 12 — Advanced Query Parameters

Now make your APIs powerful.

For:

GET /tasks

support:

status

priority

assignee

project

created_after

created_before

search

page

limit

sort

Example:

/tasks?

status=IN_PROGRESS

&priority=HIGH

&page=2

&limit=20

&sort=-created_at

Step 29. Pagination

Return:

```
{  
  "items": [],  
  "page": 2,  
  "limit": 20,  
  "total": 152,  
  "pages": 8  
}
```

Understand:

offset pagination

and eventually:

cursor pagination

Step 30. Filtering

Build reusable filtering dependencies.

You should eventually be able to do:

TaskFilter

ProjectFilter

IncidentFilter

This teaches you how to keep complex query logic organized.

PHASE 13 — Incident Management

Now build the second major domain.

Create:

incidents

Fields:

id

incident_number

title

description

severity

status

project_id

reported_by

assigned_team

started_at

resolved_at

created_at

updated_at

Step 31. Incident lifecycle

Implement:

OPEN
↓
INVESTIGATING
↓
IDENTIFIED
↓
MITIGATING
↓
RESOLVED
↓
CLOSED

Don't allow invalid transitions.

For example:

CLOSED → INVESTIGATING

should fail.

This logic belongs in your service layer.

PHASE 14 — Incident Timeline

Create:

incident_events

Every change creates an event.

Example:

Incident created
Status changed
Severity changed
Assigned user changed
Comment added
Resolved
Closed

Endpoint:

GET /incidents/{id}/timeline

PHASE 15 — Comments

Create:

comments

Allow comments on:

Tasks

Incidents

Implement:

POST

GET

PATCH

DELETE

Add:

created_by

created_at

updated_at

PHASE 16 — Mentions

Support:

@john investigate this

Parse mentions.

Create:

mentions

Then generate notifications.

This will later connect to your background worker.

PHASE 17 — Notifications

Create:

notifications

Types:

TASK_ASSIGNED
COMMENT_MENTION
INCIDENT_ASSIGNED
INCIDENT_UPDATED
INCIDENT_RESOLVED

Endpoints:

GET /notifications
PATCH /notifications/{id}/read
POST /notifications/read-all

PHASE 18 — Background Tasks

Now introduce FastAPI's `BackgroundTasks`.

Example:

POST /incidents

should:

1. Create incident
2. Return response
3. Background:
 - send notifications
 - send email
 - create audit entry

Learn exactly what `BackgroundTasks` is good for—and what it is **not** good for.

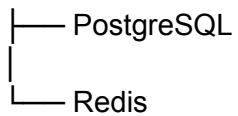
PHASE 19 — Redis

Add Redis to Docker.

Now your infrastructure becomes:

FastAPI

|



Use Redis for:

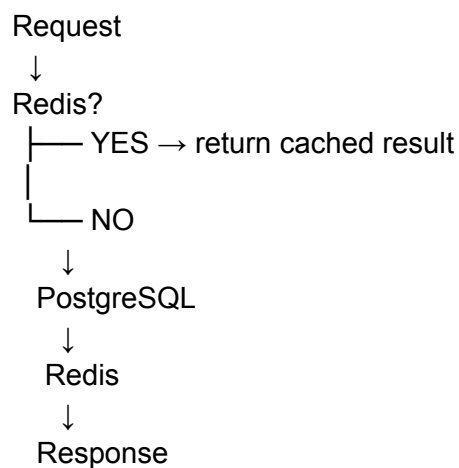
Caching
Rate limiting
Temporary data
WebSocket coordination
Job queues

PHASE 20 — Caching

Cache:

GET /analytics/dashboard

Flow:



Implement TTL.

Then learn the difficult part:

Cache invalidation.

When a task changes, determine which cache entries must be invalidated.

PHASE 21 — WebSockets

Now implement:

/ws/incidents/{incident_id}

When an incident changes:

FastAPI

↓

WebSocket Manager

↓

All connected clients

send:

```
{  
  "event": "incident.updated",  
  "incident_id": 123,  
  "status": "RESOLVED"  
}
```

Step 32. WebSocket authentication

Don't leave the WebSocket public.

Authenticate the connection.

Learn the differences between:

HTTP authentication

and:

WebSocket authentication

PHASE 22 — Connection Manager

Create:

WebSocketManager

It should manage:

```
connect()  
disconnect()  
send_personal()
```

broadcast()

You'll learn connection lifecycle management.

PHASE 23 — File Uploads

Create:

attachments

Endpoint:

POST /tasks/{id}/attachments

Use:

UploadFile
File

Validate:

file size
MIME type
extension

Don't trust the filename supplied by the client.

PHASE 24 — Object Storage

Don't permanently rely on:

./uploads

Run MinIO locally.

Architecture:

FastAPI

↓

MinIO

↓

Object

Store only metadata in PostgreSQL:

filename
size
mime_type
storage_key
uploaded_by
created_at

PHASE 25 — Middleware

Implement middleware for:

Request ID

Every request gets:

X-Request-ID

Request timing

Log:

GET /tasks → 83ms

Request logging

Record:

method
path
status
duration
request_id

Learn:

HTTP middleware

and middleware ordering.

PHASE 26 — Exception Handling

Create application exceptions:

ResourceNotFound
PermissionDenied
AuthenticationFailed
InvalidStateTransition
RateLimitExceeded

Create consistent responses:

```
{
  "error": {
    "code": "TASK_NOT_FOUND",
    "message": "Task does not exist.",
    "request_id": "req_123"
  }
}
```

Implement custom exception handlers.

PHASE 27 — HTTP Status Codes

Stop returning **200** for everything.

Learn when to use:

200 OK
201 Created
202 Accepted
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
429 Too Many Requests
500 Internal Server Error

This sounds basic, but understanding correct API semantics is important.

PHASE 28 — Rate Limiting

Use Redis.

For example:

100 requests / minute

per user.

Special limits:

/login → 5/minute

/password-reset → 3/minute

/general API → 100/minute

Return:

429 Too Many Requests

PHASE 29 — Lifespan

Use FastAPI's application lifespan functionality.

Startup:

Initialize resources

Connect Redis

Prepare services

Shutdown:

Close connections

Cleanup resources

Understand why application lifecycle management matters.

PHASE 30 — Health Checks

Create:

GET /health/live

GET /health/ready

Difference:

Liveness



Is the application alive?

Readiness



Can the application actually serve requests?

Readiness should check:

PostgreSQL

Redis

PHASE 31 — Audit Logging

Every important action becomes an audit event.

Database:

audit_logs

Fields:

id

user_id

organization_id

action

entity_type

entity_id

old_value

new_value

ip_address

user_agent

created_at

Examples:

CREATE_PROJECT

DELETE_TASK

CHANGE_INCIDENT_STATUS

ADD_MEMBER

REMOVE_MEMBER

PHASE 32 — Analytics

Build:

GET /analytics/dashboard

Return:

total users
total projects
total tasks
completed tasks
open incidents
critical incidents
average resolution time
tasks by status
tasks by priority
incidents by severity

Use SQL aggregation.

Then cache the result using Redis.

Now you've combined:

FastAPI
+
SQLAlchemy
+
PostgreSQL
+
Redis

PHASE 33 — Streaming

Add an endpoint such as:

GET /incidents/{id}/export

that streams a large CSV/report rather than loading everything into memory.

Learn:

StreamingResponse

This is useful for understanding how HTTP responses can be generated incrementally.

PHASE 34 — Dependency Overrides

This is especially important for testing.

Your application might normally have:

```
get_db()
```

But tests should use:

```
test_database
```

Override the dependency.

Learn:

```
app.dependency_overrides
```

This is one of FastAPI's excellent testing features.

PHASE 35 — Testing

Now test everything.

Create:

```
tests/  
├── unit/  
├── integration/  
└── api/
```

Unit tests

Test:

```
password hashing  
JWT generation  
permission logic  
state transitions  
pagination  
services
```

API tests

Test:

POST /auth/register

POST /auth/login

GET /users/me

POST /projects

POST /tasks

PATCH /tasks/{id}

Authorization tests

These are particularly important.

Test:

Member cannot delete project

Viewer cannot modify task

User cannot access another organization

Unauthorized user cannot access private API

Validation tests

Test:

invalid email

invalid priority

missing title

negative page

huge limit

invalid UUID

invalid datetime

PHASE 36 — Async Testing

Use:

pytest-asyncio

httpx.AsyncClient

Test asynchronous endpoints.

Understand:

async def
await
AsyncSession
AsyncClient

PHASE 37 — WebSocket Testing

Test:

connect
authenticate
receive event
broadcast event
disconnect

Verify that:

Client A updates incident



Client B receives event

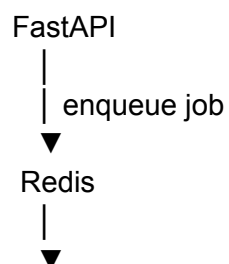
PHASE 38 — Background Workers

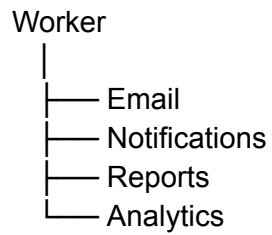
Now graduate from FastAPI's simple [BackgroundTasks](#).

Introduce:

Redis
+
Celery/RQ/Dramatiq

Architecture:





Learn the distinction between:

BackgroundTasks

and:

Dedicated task queues

PHASE 39 — Email

Create an abstraction:

EmailService

Implement:

ConsoleEmailService

first.

Then optionally:

SMTP

SES

SendGrid

Emails:

Welcome

Verify email

Password reset

Task assignment

Incident notification

Incident resolution

PHASE 40 — API Versioning

Structure:

/api/v1/

Later create:

/api/v2/

Don't break v1 when v2 is introduced.

Understand how API evolution works.

PHASE 41 — OpenAPI Mastery

Now revisit:

/docs

Make your API documentation professional.

Every endpoint should have:

Summary

Description

Tags

Request model

Response model

Status codes

Error responses

Examples

Authentication requirements

Your OpenAPI specification should be good enough that someone else can build a frontend without reading your source code.

PHASE 42 — Configuration

Use:

pydantic-settings

Create:

Settings

Variables:

DATABASE_URL
REDIS_URL
JWT_SECRET
JWT_EXPIRATION
ENVIRONMENT
CORS_ORIGINS
STORAGE_ENDPOINT

Have:

.env
.env.example

Never hardcode secrets.

PHASE 43 — CORS

Configure CORS for your frontend.

Understand:

Origin
Methods
Headers
Credentials
Preflight requests

Don't simply use:

```
allow_origins=["*"]
```

in production when credentials are involved.

PHASE 44 — Security Review

Now deliberately attack your own API.

Try:

Access another organization's task
Change another user's role

Delete another user's project
Use expired JWT
Use invalid JWT
Reuse revoked refresh token
Upload malicious file
Send huge request
Spam login
Guess resource IDs

Fix everything you discover.

This phase is more valuable than adding another feature.

PHASE 45 — Database Optimization

Use PostgreSQL indexes.

Find slow queries.

Investigate:

EXPLAIN ANALYZE

Optimize:

tasks(project_id)
tasks(assignee_id)
tasks(status)
incidents(status)
incidents(severity)
audit_logs(organization_id)

Learn why indexes help and when they hurt.

PHASE 46 — Transactions

Introduce operations that require multiple database changes.

Example:

Resolve incident



Update incident

↓
Create timeline event
↓
Create audit record
↓
Create notification

If one operation fails, understand when everything should roll back.

Learn database transactions properly.

PHASE 47 — Dockerize Everything

Your final `docker-compose.yml` should contain approximately:

services:

api
postgres
redis
worker
minio

Your API should start with something like:

`docker compose up`

PHASE 48 — Production Docker Image

Create a proper:

Dockerfile

Learn:

base images
dependency installation
environment variables
non-root users
port exposure
health checks

PHASE 49 — Logging

Implement structured logs.

Instead of:

Something went wrong

produce:

```
{  
  "level": "ERROR",  
  "request_id": "req_123",  
  "method": "POST",  
  "path": "/tasks",  
  "status": 500,  
  "duration_ms": 124,  
  "user_id": 42  
}
```

PHASE 50 — Observability

Track:

Request count

Error count

Latency

Database latency

Redis latency

Background jobs

WebSocket connections

You can optionally add:

Prometheus

Grafana

OpenTelemetry

This moves the project toward production engineering.

PHASE 51 — Final API

At the end, your API might look like:

```
/api/v1
├── /auth
│   ├── register
│   ├── login
│   ├── refresh
│   ├── logout
│   └── forgot-password
├── /users
│   ├── me
│   └── {id}
├── /organizations
│   ├── ...
│   └── {id}/members
├── /projects
│   └── {id}
├── /tasks
│   └── {id}
├── /incidents
│   └── {id}
├── /comments
├── /notifications
├── /attachments
├── /analytics
├── /audit-logs
└── /health
```

And:

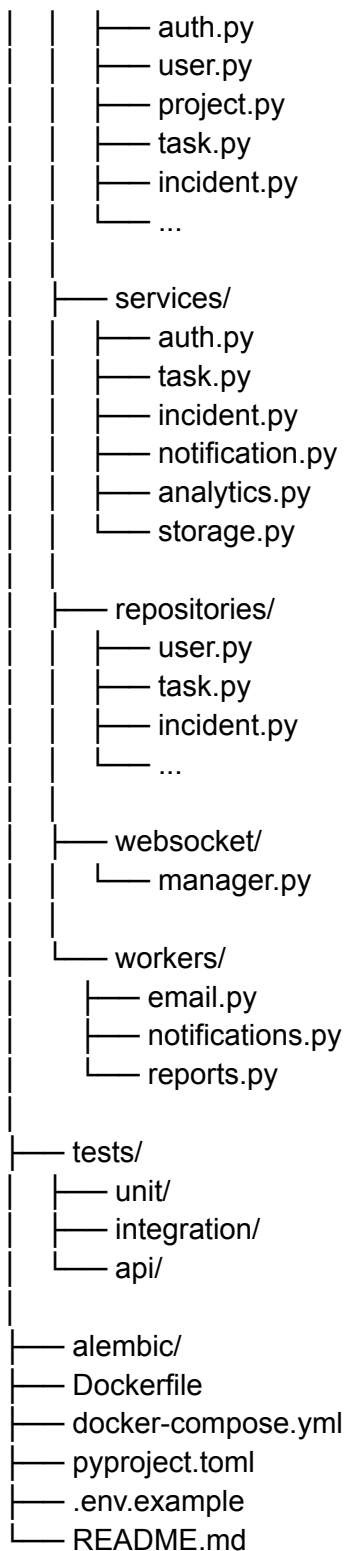
```
/ws/incidents/{id}
```

for real-time communication.

PHASE 52 — Final Architecture

Your finished application should roughly look like:

```
pulseboard/
├── app/
│   ├── main.py
│   ├── api/
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── users.py
│   │       ├── organizations.py
│   │       ├── projects.py
│   │       ├── tasks.py
│   │       ├── incidents.py
│   │       ├── comments.py
│   │       ├── notifications.py
│   │       ├── attachments.py
│   │       ├── analytics.py
│   │       └── audit.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── logging.py
│   │   ├── exceptions.py
│   │   └── middleware.py
│   ├── db/
│   │   ├── database.py
│   │   ├── base.py
│   │   └── migrations/
│   ├── models/
│   │   ├── user.py
│   │   ├── organization.py
│   │   ├── project.py
│   │   ├── task.py
│   │   ├── incident.py
│   │   ├── comment.py
│   │   ├── notification.py
│   │   └── audit.py
│   └── schemas/
```



What You Will Actually Learn

The important thing is that this isn't simply a list of features.

By the end, you should be able to explain these concepts **without relying on memorized tutorials**:

FastAPI fundamentals

FastAPI application
APIRouter
Path parameters
Query parameters
Request bodies
Response models
Status codes
Headers
Cookies
Forms
File uploads

Pydantic

BaseModel
Field
Validation
Serialization
Nested models
Enums
Custom validators
Settings

Dependency Injection

Database dependency
Authentication dependency
Authorization dependency
Reusable dependencies
Nested dependencies
Dependency overrides

Security

OAuth2
JWT
Password hashing
Access tokens
Refresh tokens
RBAC
Multi-tenancy
CORS
Rate limiting

Async

async def
await
AsyncSession
AsyncClient
WebSockets
Background work

API architecture

Routers
Services
Repositories
Schemas
Models
Exception handlers
Middleware

Database

PostgreSQL
SQLAlchemy
Relationships
Transactions
Indexes
Migrations
Alembic
Query optimization

Infrastructure

Docker
Redis
Object storage
Background workers
Caching
Queues

Production

Logging
Health checks
Observability
Testing
Configuration
Security
Performance
API versioning

The Most Important Rule

Do not build Phase 1 → Phase 52 as a giant project from day one.

Build it incrementally.

Your development loop should be:

LEARN ONE CONCEPT



IMPLEMENT IT



BREAK IT



DEBUG IT



WRITE TESTS



REFACTOR IT



MOVE ON

For example:

Learn dependencies



Implement `get_db()`



Implement `get_current_user()`



Implement `require_role()`



Write tests



Break authorization intentionally



Fix it



Move on

That will teach you **far more FastAPI** than simply following a tutorial and copying code.

Your Final Milestone

When you finish, you should be able to clone the project onto a new machine and run:

docker compose up

Then:

<http://localhost:8000/docs>

and see a fully documented API.

You should be able to:

Register



Login



Create Organization



Invite Users



Create Project



Create Tasks



Assign Tasks



Report Incident



Collaborate through comments



Receive notifications



Watch incident update through WebSocket



Upload attachments



Resolve incident

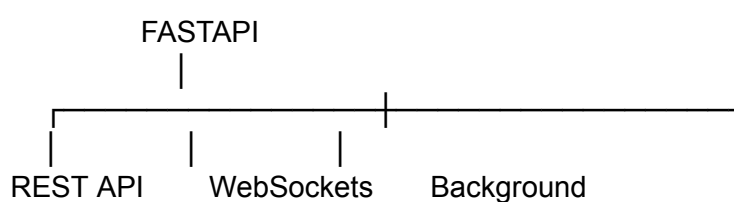


View analytics



Inspect audit history

while the backend simultaneously demonstrates:





That is the project I would use if your actual objective is to become very strong at FastAPI rather than merely learn how to create endpoints.